Applied ML Final

90803B-ML Foundations

Eugene Ho, Matthew Lawlor, Jonah Zembower

Date: April 30, 2026

Executive Summary

Discarded needles on Boston streets are a recurring public health hazard with real costs such as needle-stick exposures, loss of community-trust, and unevenly distributed cleanup response times. The Boston Public Health Commission triage these incidents through the city's 311 system, but they respond after a needle is reported, not before clusters form. Our project asks whether the city's own data is enough to predict where and when needle pickup activity will surge and how we can then deploy proactively.

We trained a Temporal Convolutional Neural Network (Temporal CNN) on Boston 311 service-request data to predict weekly spikes in neighborhood-level needle pickup activity. On a 2024-2025 holdout covering all 24 Boston Neighborhoods, the model can be tuned to catch 100% of true spikes at about 104 alerts per month, 84% at about 68 alerts per month (our recommended balanced default), or 33% at about 27 alerts per month for a tight resource constrained operation. The neural-network track moved through three iterations, replacing a flat MLP that failed with a recall of .072 with a 1D convolutional architecture that operates on each neighborhood's recent 8-week history, sweeping the spike-label sensitivity, and tuning the decision threshold.

Intro and Policy Context

Boston, like many U.S. cities, has spent the last decade absorbing the operational consequences of the opioid epidemic. Discarded syringes turn up in parks, sidewalks, and residential areas and as our data shows, the problem is more geographically distributed than what press coverage may suggest. Cleanup is currently dissipated through Boston's 311 system, which is reactive by design. That model has two practical failure models. In areas where 311 use is lower, due to things such as language access, digital divide, and distrust in city services, needles linger longer. And even in high-reporting neighborhoods, surges are not predicted ahead of time so screws and harm-reduction outreach workers cannot pre-position. Both are equity issues, not just efficiency issues.

Our policy question asks: Can publicly available 311 data be used to predict, in advance, the neighborhoods in Boston that will experience high needle pickup activity accurately enough to support proactive deployment of cleanup crews and outreach? We are not looking to find a single accuracy number, but a tool that supports two real decisions: management planning next month's staffing, and a public-health team deciding where outreach should show up in the next week. It is a tool that helps a public health team ask "given last week's activity, here are the neighborhoods where outreach should show up next week". The framing matters as well since this project is designed to support a deployment decision and not to predict a hidden attribute about the individual people who live in those neighborhoods.

Data and Methodology

We worked from a cleaned Boston 311 extract covering January 2017 through late 2025, which contains roughly 71,000 needle related records with full latitude and longitude coverage, The multi-year span gives the model enough history to learn seasonality and stable per-neighborhood baseline, and the weekly volume is sufficient to train both classical and sequence-based learners. We aggregated to a balanced neighborhood-week panel that explicitly includes zero-incident weeks. Including those zero weeks is

important as spike detection is meaningless without the contrast of normal weeks, and dropping them would bias the panel toward areas that already show activity.

The prediction target is a binary spike indicator expressed as the following:

$$\text{is\_spike}_t = \begin{cases} 1, & \text{if needle\_count}_t > \text{rolling\_mean}_{8w,t} + \text{SPIKE\_STD\_MULT} \cdot \text{rolling\_std}_{8w,t} \\ 0, & \text{otherwise} \end{cases}$$

A week is labeled a spike if that week's count is above the neighborhood's 8-week rolling average plus SPIKE_STD_MULT times the 8-week rolling standard deviation.
We tested SPIKE_STD_MULT values like 1.0, 1.25, 1.5, 1.75, 2.0, 2.5 (with 1.5 as the common baseline).

Justification: Why This Was Done
Before conducting model tuning, we performed a label sensitivity analysis to understand how changes in the outcome definition would affect the modeling process. This step was necessary because the label definition directly influences the prevalence of the positive class, the severity of class imbalance, apparent model performance, and the expected downstream alert volume.

In this context, the label parameter is not only a technical modeling choice. It also functions as a policy decision that determines which patients or cases are counted as positive events. As a result, changing the event definition can substantially alter how models appear to perform and how many alerts the system would generate in practice.

For this reason, we treated label sensitivity testing as an important step before comparing or tuning models. Without this analysis, model comparisons could be misleading because observed differences in performance might reflect changes in the outcome definition rather than true improvements in predictive ability.

Results from initial exploration
In our initial exploration, we compared two models for spike detection: a logistic regression baseline and a neural network. The logistic regression model was implemented with feature scaling and class_weight="balanced", while the neural network used an MLPClassifier with two hidden layers of 64 and 32 neurons. Evaluation was conducted on a test set of 2,520 observations, including 291 true spike events.

The logistic regression baseline achieved a precision of 0.138, recall of 0.632, and F1-score of 0.226. Its confusion matrix showed 1,078 true negatives, 1,151 false positives, 107 false negatives, and 184 true positives. Although its overall accuracy was relatively low (0.501), the model identified a substantial proportion of spike events, which is important in an alerting context where missed spikes are costly.

By contrast, the neural network achieved a higher precision of 0.375 but a much lower recall of 0.072, with an F1-score of 0.121. While its overall accuracy was high (0.879), this result appears to be driven largely by correct classification of the majority non-spike class. In practical terms, the model missed most spike events, limiting its usefulness for spike alerting.

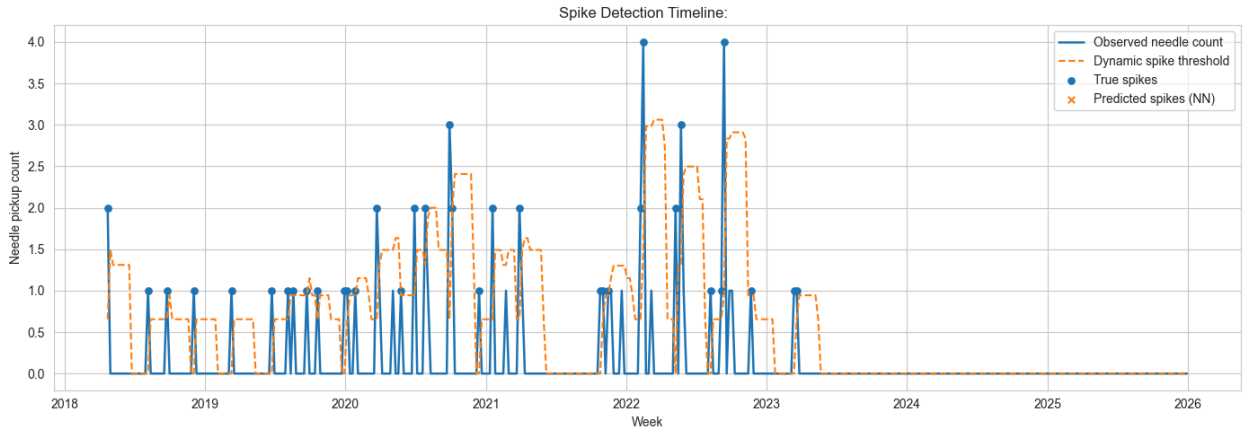

Figure 1: Weekly Spike Detection Timeline

Figure 1 shows the weekly observed needle counts, the dynamic spike threshold, true spikes, and MLP-predicted spikes. The timeline indicates a strongly imbalanced pattern: most weeks are non-spikes, with only occasional spike events.

This visual pattern aligns with the model results. The MLP predicts few spikes at true spike periods, which is consistent with its low recall (0.072), despite high overall accuracy (0.879). The high accuracy is therefore largely driven by correct non-spike predictions.

Compared with the MLP, logistic regression better supports the alerting objective because it captures many more true spikes (recall = 0.632), even with more false positives. Overall, both the metrics and the timeline suggest that logistic regression is the more suitable model for spike detection at this stage.

Next Steps

Because the MLP neural network did not perform adequately, the project proceeded with a focused set of next steps to improve spike detection performance and policy relevance.

First, we will replace the MLP with sequence-based models, using a Temporal CNN architecture. The MLP produced very low recall (0.072) and generated a convergence warning, indicating that it is not capturing temporal spike behavior effectively. Because spike events are inherently time-dependent and may involve bursts, lead-up trends, and lagged effects, sequence models provide a more appropriate framework for learning these dynamics.

Second, we tuned spike-threshold sensitivity by adjusting SPIKE_STD_MULT. The current spike label is defined by a rule (rolling_mean_8 + 1.5 * rolling_std_8), so this threshold directly determines how many cases are labeled as spikes before model training begins. Since spikes currently represent only about 11.9% of observations, even modest threshold changes can meaningfully alter class balance, model learning conditions, recall, and false-alarm volume. This makes threshold tuning the most direct lever for managing the recall–false alarm trade-off.

Third, we will expand feature engineering to include spatial lag variables from adjacent neighborhoods. This step is intended to capture potential hotspot spillover effects, where elevated activity in one area may precede or coincide with spikes in nearby areas. Incorporating spatial context should improve the model's ability to detect localized, interconnected risk patterns.

Finally, model performance will be evaluated using operationally meaningful outcomes, specifically "alerts per month" and "detected true spikes," alongside standard statistical metrics. This is necessary because conventional metrics can be misleading in imbalanced settings, as shown by the MLP's high overall accuracy despite poor spike detection. For policy and deployment decisions, stakeholders need to understand expected investigation workload and true event capture, not only abstract model diagnostics.

Temporal CNN Workflow and Results

We implemented a Temporal CNN on the neighborhood-week panel (including zero-incident weeks) and kept the spike definition consistent with earlier models: spike_threshold = rolling_mean_8 + SPIKE_STD_MULT * rolling_std_8, with is_spike as the target. Each sample used the prior 8 weeks of needle_count (SEQ_LEN = 8) to predict the current week's spike status. Data were split by time (train < 2024-01-01, test >= 2024-01-01) to avoid leakage and remain comparable with prior results. Because spikes are the minority class, class weights were applied during training. The model used two causal Conv1D layers, global max pooling, and a dense classification head, with early stopping on validation recall.

The key finding is that the Temporal CNN is much more effective at capturing spikes than previous models, but at the cost of substantial over-alerting. On the test set, it achieved recall of 0.880 (256/291 spikes detected), precision of 0.139, and F1-score of 0.240, with confusion matrix [[644, 1585], [35, 256]]. Relative to logistic regression, recall improved strongly (0.632 to 0.880), precision remained nearly unchanged (0.138 to 0.139), and F1 increased slightly (0.226 to 0.240). However, false positives increased considerably, reducing overall accuracy.

Overall, the Temporal CNN functions as an aggressive alert model: it minimizes missed spikes but generates high alert volume. This makes threshold calibration and operational metrics (e.g., alerts per month and detected true spikes) the critical next step.

Threshold Tuning: Method, Findings, and Policy Implications

We tested how spike-label strictness (SPIKE_STD_MULT = 1.0 to 2.5) affects predictive performance. For each value, we rebuilt spike labels, reran the same feature pipeline, used the same time-based train/test split, trained the baseline logistic model, and compared precision, recall, F1, and alert counts. This isolated the effect of label policy before further CNN tuning.

The key finding is a clear trade-off: as the threshold increased, spike prevalence dropped (overall 17.34% to 5.52%), recall rose (0.596 to 0.733), but precision fell sharply (0.211 to 0.080) and F1 declined (0.312 to 0.145). Alert volume stayed high across settings (about 1,193 to 1,348), meaning stricter labels did not substantially reduce operational burden. The best balanced setting was SPIKE_STD_MULT = 1.0

(highest F1), while 2.5 maximized recall at the cost of heavy false alarms.

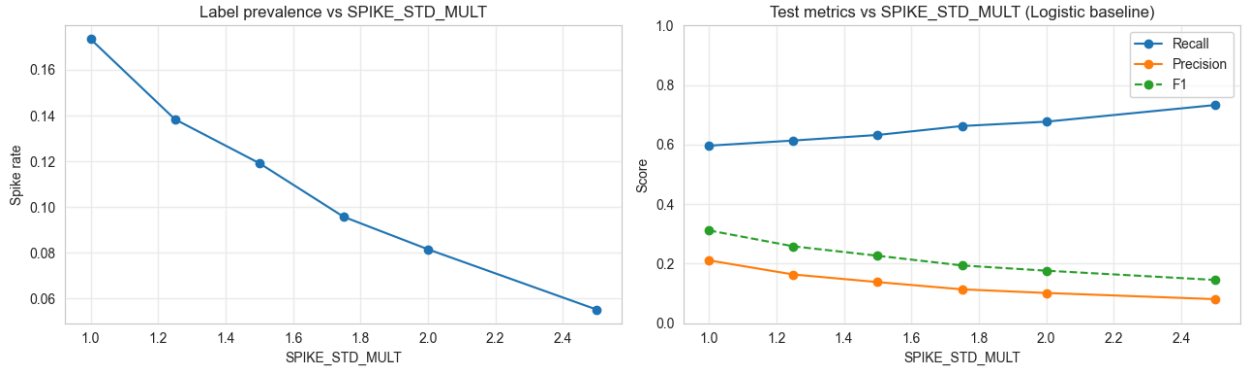

Figure 2: Sensitivity of Spike Label Threshold and Prediction Performance

Figure 2 shows how tightening the spike definition by increasing SPIKE_STD_MULT systematically reduces how often weeks are labeled as spikes as described above, but it also shifts model behavior in an important way. As spikes become rarer, the model captures a larger share of those labeled spikes (higher recall) while becoming less reliable when it issues an alert (lower precision), which causes F1 to decline.

In practical terms, this reflects a classic sensitivity trade-off: stricter labels may improve spike capture rates, but they also produce noisier alerting performance. For policy use, the figure suggests that spike-threshold selection should be based on operational priorities—whether the goal is maximizing detection or maintaining a manageable and credible alert stream.

For the policy question, this implies that publicly available 311 data does show predictive signal and can support advance identification of higher-risk neighborhoods, but "accurate enough" depends on operational tolerance for false positives. The data are most useful as a proactive triage tool (prioritizing where to deploy crews/outreach) rather than as a high-precision forecasting system. To make it policy-ready, the model should be selected and tuned against deployment targets (e.g., alerts per month and true spikes detected), not accuracy alone.

Comparative Results and Transition to Risk Classification

The comparative analysis shows that the tuned Temporal CNN outperforms the logistic baseline in practical spike detection, especially in recall-priority and balanced settings. Relative to logistic regression (precision 0.074, recall 0.756, F1 0.135), the CNN improves detection quality at operationally meaningful thresholds: at 0.20, it captures all spikes (recall 1.000) but produces very high workload (104.3 alerts/month); at 0.55, it provides the strongest overall trade-off (precision 0.149, recall 0.835, F1 0.253, 67.8 alerts/month); and at 0.65, it lowers workload (27.3 alerts/month) but misses many spikes (recall 0.333; 8.1 missed spikes/month).

These results indicate that thresholding is a policy control, not just a technical setting: more alerts reduce missed spikes, while fewer alerts increase missed events. The most defensible default is the balanced CNN (0.55), with threshold adjustment based on staffing capacity and risk tolerance.

This directly motivates the next section on risk classification. After establishing how model thresholds translate into workload-versus-miss trade-offs, the workflow shifts to identifying which neighborhoods should be prioritized as high risk under those operational constraints. In other words, spike forecasting defines when and how aggressively to alert, while risk classification defines where to target resources most effectively.

Risk Classification Analysis:
In the midterm project, we defined the top quarter of high probability needle cases as high risk while the lower part being low risk. This was calculated by using the quantile of the 75th percentile in the data for needle_counts of all locations. Our goal was to classify this with a model to interpret predictability for the most high risk areas while maximizing on the recall. Therefore, we could ensure that funds each month would go to those locations we are sure of being high risk. This established some general overall success using a Random Forest Classification (threshold =0.3) with the following results below:

Metric	Score
Accuracy	0.875
Precision	0.804
Recall	0.760
F1 Score	0.782
ROC-AUC	0.944

Table 1: Random Forest Classification Metrics

However, we realized there was some overfitting as the training performed much better, and there was some clear need for improvement. Therefore, we decided to try new models and then compute a voting ensemble model that performed the best in generalizability for the test data out of all of them. As per the project, we ensured the use of a neural network in one of these models called TabNet. We chose this model because it works better for small tabular data. Now, we will begin to discuss the following models that were used and their performance. We tried to maintain the similar thresholds for each of 0.3 similar to the random forest above to compare.

XGBoost
We used a gradient boosted tree because it is typically the strongest of these models in this type of data. We also tried to maintain the scale position weight to help work against the class imbalance. We also wanted to do early stopping regularization for reduced overfitting of these models. For the test results, we obtained:

Metric	Value
Accuracy	0.873
Precision	0.852

Recall	0.689
F1 Score	0.762
ROC-AUC	0.935

Table 2: XGBoost Metrics

TabNet

For TabNet, the goal was to choose a neural network that would work better for the tabular based data. This uses sequential attention to pick which features to reason over at each decision step. Here, the precision didn't perform quite as well and the test results even showed a larger focus related to recall, which we intended, but with less generalization to other accuracy metrics. The test results are showcased below:

Metric	Value
Accuracy	0.787
Precision	0.583
Recall	0.964
F1 Score	0.727
ROC-AUC	0.950

Table 3: TabNet Metrics

CatBoost

Finally, it made sense for us to also try CatBoost in the modeling approaches. Since it is a gradient-boosted tree library, it offers better performance typically. Furthermore, it also does ordered boosting, which will reduce potential overfitting from target leakage during the gradient estimation. Also, it has symmetric trees, which act as a strong regularizer. Because this model learned so quickly in the training set, we decided to use a technique of validation tuning rather than the training tuning, which will be a straight line across different thresholds. For this method, the results were:

Metric	Value
Accuracy	0.884
Precision	0.822
Recall	0.772
F1 Score	0.796
ROC-AUC	0.937

Table 4: CatBoost Metrics

Side-by-Side Comparison

For each of these models, we wanted to perform a simple side-by-side comparison that would showcase the differences in accuracy metrics and the related results that all performed relatively well:

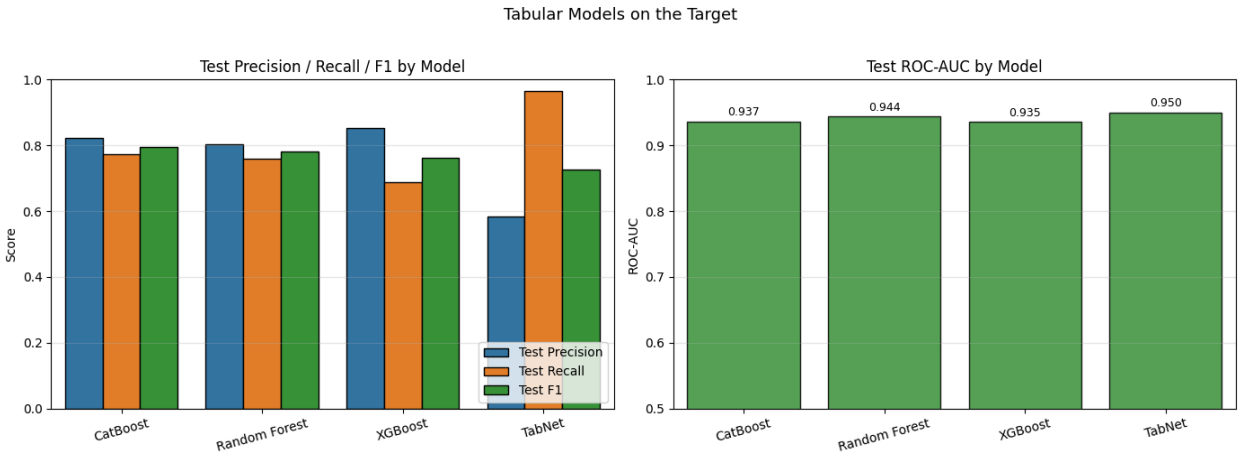

Figure 3: Comparative Analysis

Each of the models above had their own strengths and weaknesses. There wasn't a clearly defined winner even focusing on recall, because TabNet lost a lot in other metrics. Therefore, we wanted to try an Ensembling voting method that would characterize well across these different model metrics.

<u>Ensembling</u>

Since all of these models produced a per-row probability on the same train/test split and target, we could blend their predictions together. So, we decided to do voting strategies that made sense for different contexts. First, soft voting: Average each model's P(high_risk) = 1 then applying a similar threshold and outputting the results. Next, hard voting: here we are counting for a somewhat majority of the 4 models meaning at least 2 votes (2 models) would need to flag a row as high_risk for it to be classified as such. After calculating the results, we can output them with the single model alone as a comparison:

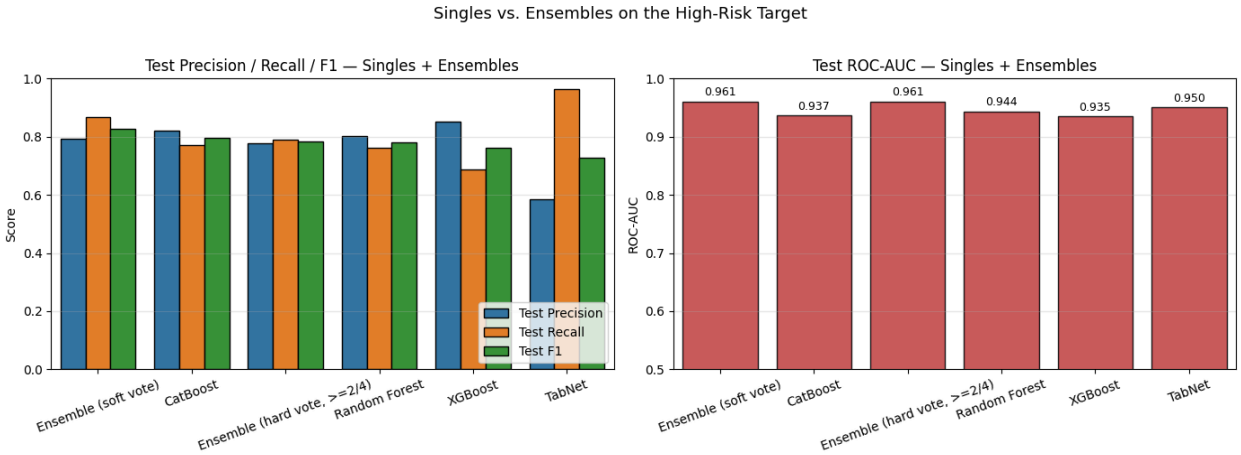

Figure 4: Comparison of Singles vs Ensembles on High Risk Targets

In the above graph, it can be shown that the soft vote ensemble method appeared to be the best approach by averaging across the models and still performing best in the recall metric. We can assume that this model would be the best approach in practice by using each model's own characteristics to predict

whether a location is of high risk or not. Here are some further images related to the explainability of those results:

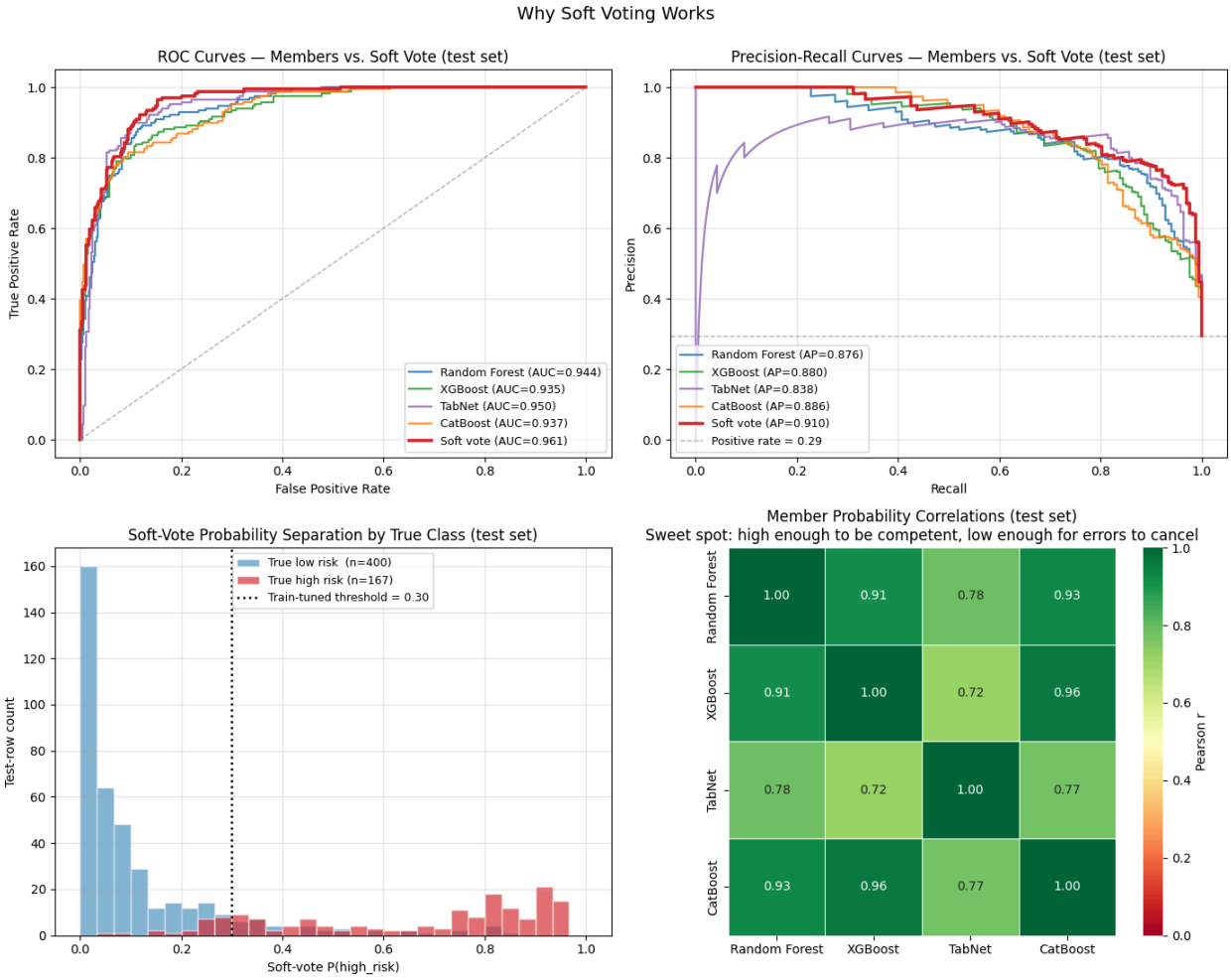

Figure 5: Explainability Visualisations

The above graphs showcase the ROC-AUC being the best in the ensemble method in the top left. Next, we see the Precision-Recall Curve where it still performs better than the rest with the highest average precision. For the last two graphs we can see the test set results and how it is generally classified with our continual 0.3 threshold. Finally, there are Pearson correlations between the models in the test set that help us understand how it is making decisions and hopefully cancelling out the errors of the other models.

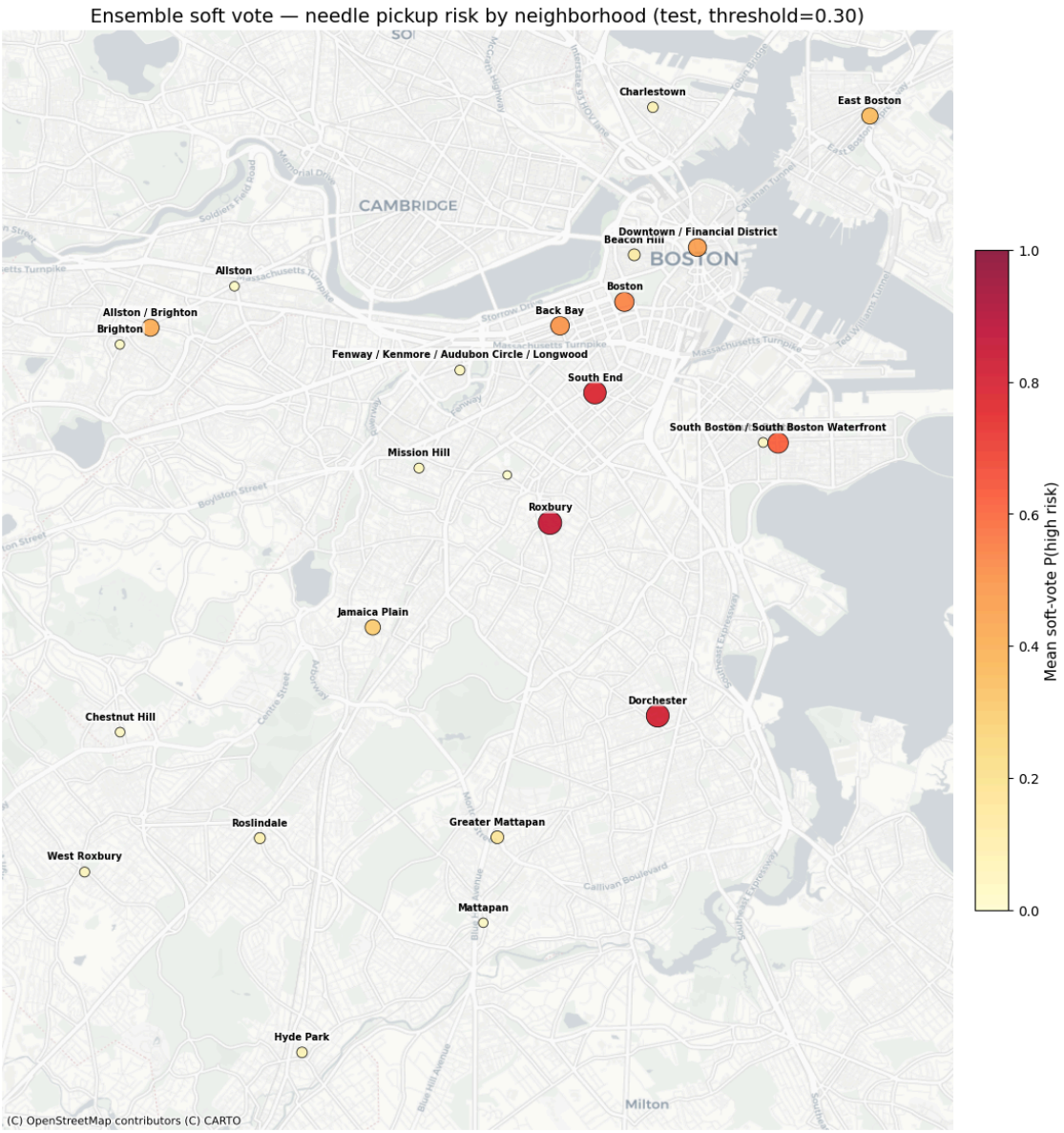

Figure 6: Neighborhood level Risk Predictions from Soft Voting Ensemble

This map above visualizes neighborhood-level risk predictions from the soft-voting ensemble on the test set using a 0.30 classification threshold. Each point represents a Boston neighborhood, and the color gradient from lighter yellow to darker red indicates increasing predicted likelihood of high needle-pickup activity (upper-quartile risk class). The figure provides a spatial interpretation of the model results by showing where risk is concentrated, helping translate model output into actionable geographic priorities for proactive cleanup and outreach.

Conclusion

The project produces two distinct but complementary findings, because the two analyses use different target definitions and therefore answer different parts of the policy problem.

In the spike detection analysis, the target is different: spikes are defined dynamically using a rolling baseline rule (rolling_mean + SPIKE_STD_MULT * rolling_std). This analysis evaluates short-term alerting performance and operational workload. Compared with logistic regression (precision 0.074, recall 0.756, F1 0.135), the tuned Temporal CNN improves detection quality at relevant operating points. At threshold 0.20, recall reaches 1.000 but alert volume is very high (104.3 alerts/month). At 0.55, performance is most balanced (precision 0.149, recall 0.835, F1 0.253, 67.8 alerts/month). At 0.65, alert burden drops (27.3/month) but missed spikes rise sharply (recall 0.333; 8.1 missed spikes/month). This demonstrates that thresholding is an operational policy lever: fewer misses require higher workload, while tighter alert budgets increase missed events.

In the risk classification analysis, the target is binary high risk defined as the upper quartile of neighborhood needle-count totals (75th percentile rule). Under this framing, the models show strong discrimination and useful generalization. The Random Forest baseline achieved accuracy 0.875, precision 0.804, recall 0.760, F1 0.782, and ROC-AUC 0.944. Follow-up models showed different trade-offs (for example, TabNet recall 0.964 with lower precision), and the soft-voting ensemble provided the most stable overall policy-facing performance, with recall around 0.868. Substantively, this means the model can identify most neighborhood-months that truly fall into the highest-need group, which supports geographic prioritization of funding and outreach.

Taken together, these results explicitly answer the policy question with a qualified yes: publicly available 311 data can be used to predict high-need neighborhoods in advance well enough to support proactive deployment, provided deployment is governed by threshold policy and team capacity. The risk classifier identifies where chronic high need is likely, while the spike model informs when to escalate response intensity. Therefore, the evidence supports using 311-based modeling as a decision-support system for proactive cleanup and outreach, rather than as a fully automated, zero-false-alarm trigger.